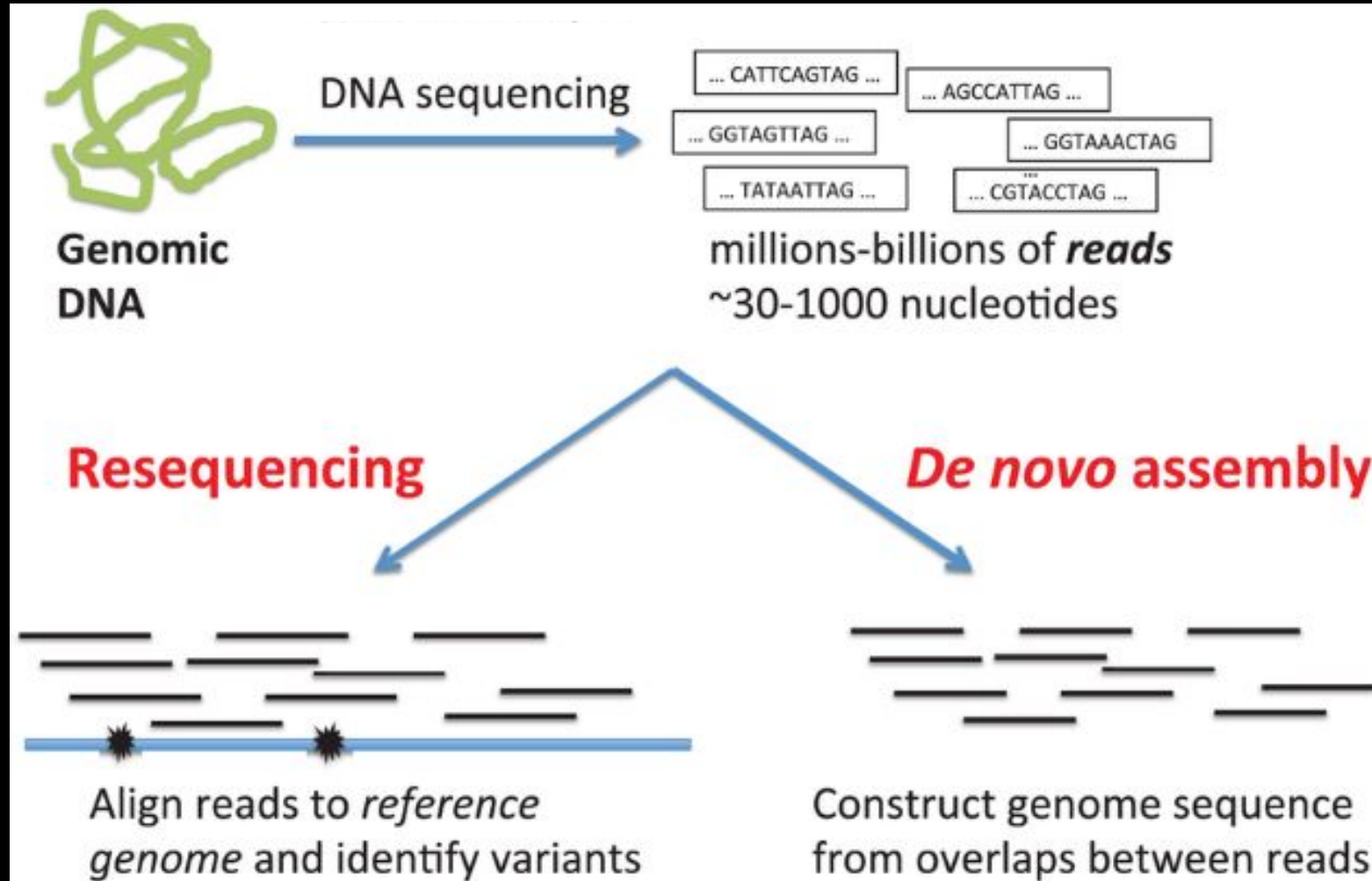


Comparing Highly Similar Sequences: Suffix Trees, Suffix Arrays and the Burrows-Wheeler Transform

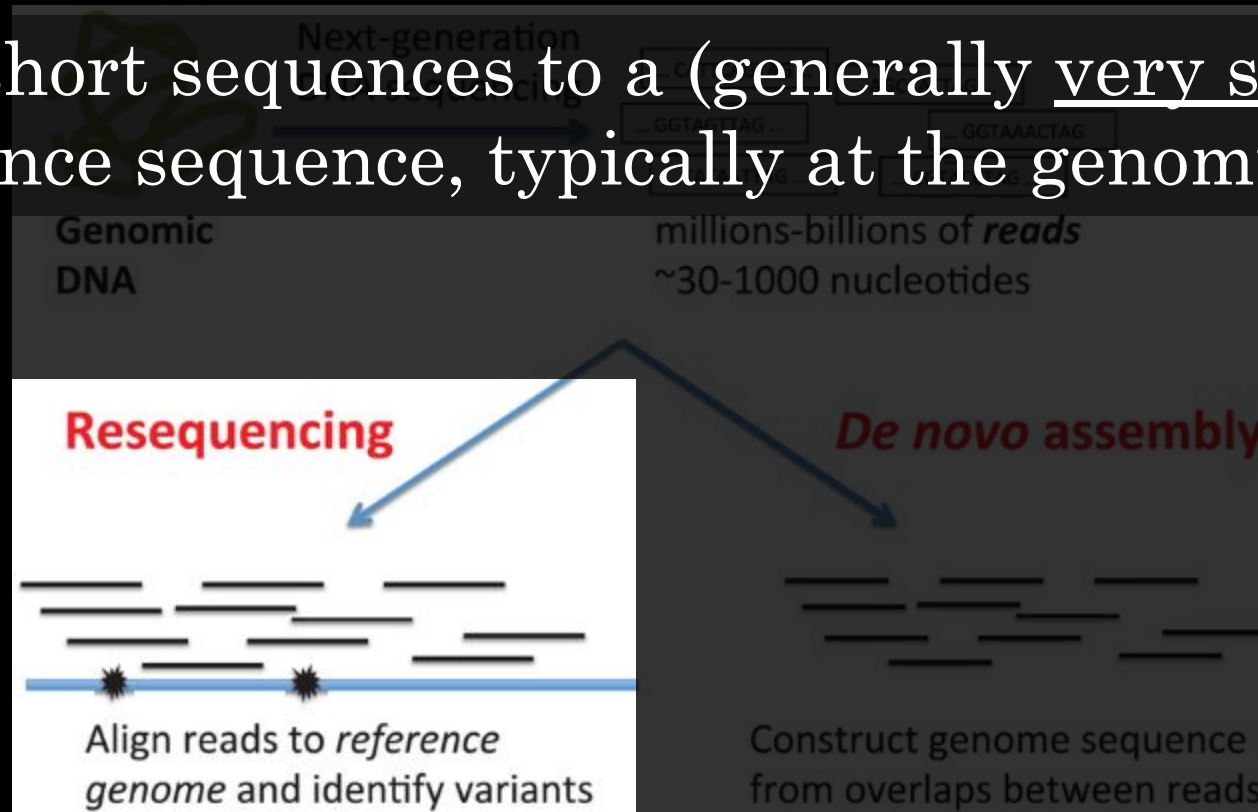


Resequencing: A Different(-ish) Use Case



Resequencing: A Different(-ish) Use Case

Map short sequences to a (generally very similar) reference sequence, typically at the genomic level



Contrasting Cases

- MEGABLAST: **Somewhat similar** sequences
- BLAST: **Remote** homology – lots of sequence differences, possibly a lot of gaps too
- PSI-BLAST / hidden Markov models: **Super remote** homology
- Mapping: **Very similar**, few to no differences / gaps

Resequencing



UK

100K

RARE GENETIC VARIANTS IN HEALTH AND DISEASE

2001

articles

Initial sequencing and analysis of the human genome

International Human Genome Sequencing Consortium*

* A partial list of authors appears on the opposite page. Affiliations are listed at the end of the paper.

The human genome holds an extraordinary trove of information about human development, physiology, medicine and evolution. Here we report the results of an international collaboration to produce and make freely available a draft sequence of the human genome. We also present an initial analysis of the data, describing some of the insights that can be gleaned from the sequence.

THE HUMAN GENOME

The Sequence of the Human Genome

J. Craig Venter,^{1*} Mark D. Adams,¹ Eugene W. Myers,¹ Peter W. Li,¹ Richard J. Mural,¹ Granger G. Sutton,¹ Hamilton O. Smith,¹ Mark Yandell,¹ Cheryl A. Evans,¹ Robert A. Holt,¹ Jeannine D. Gocayne,¹ Peter Amanatides,¹ Richard M. Ballew,¹ Daniel H. Huson,¹ Jennifer Russo Wortman,¹ Qing Zhang,¹ Chinnappa D. Kodira,¹ Xiangqun H. Zheng,¹ Lin Chen,¹ Marian Skupski,¹ Gangadharan Subramanian,¹ Paul D. Thomas,¹ Jinghui Zhang,¹ George L. Gabor Miklos,² Catherine Nelson,³ Samuel Broder,¹ Andrew G. Clark,⁴ Joe Nadeau,⁵ Victor A. McKusick,⁶ Norton Zinder,⁷ Arnold J. Levine,⁷ Richard J. Roberts,⁸ Mel Simon,⁹ Carolyn Slayman,¹⁰ Michael Hunkapiller,¹¹ Randall Bolanos,¹ Arthur Delcher,¹ Ian Dew,¹ Daniel Fasulo,¹ Michael Flanigan,¹ Liliana Florea,¹ Aaron Halpern,¹ Sridhar Hannenhalli,¹ Saul Kravitz,¹ Samuel Levy,¹ Clark Mobarry,¹ Knut Reinert,¹ Karin Remington,¹ Jane Abu-Threideh,¹ Ellen Beasley,¹ Kendra Biddick,¹ Vivien Bonazzi,¹ Rhonda Brandon,¹ Michele Cargill,¹ Ishwar Chandramouliswaran,¹ Rosane Charlab,¹ Kabir Chaturvedi,¹ Zuoming Deng,¹ Valentina Di Francesco,¹ Patrick Dunn,¹ Karen Eilbeck,¹ Carlos Evangelista,¹ Andrei E. Gabrielian,¹ Weiniu Gan,¹ Wangmao Ge,¹ Fangcheng Gong,¹ Zhiping Gu,¹ Ping Guan,¹ Thomas J. Heiman,¹ Maureen E. Higgins,¹ Rui-Ru Ji,¹ Zhaoxi Ke,¹ Karen A. Ketchum,¹ Zhongwu Lai,¹ Yiding Lei,¹ Zhenya Li,¹ Jiayin Li,¹ Yong Liang,¹ Xiaoying Lin,¹ Fu Lu,¹ Gennady V. Markov,¹ Natalia Milas,¹ Helen M. Moore,¹ Ashwin Kumar K. Nair,¹

2022

HUMAN GENOMICS

The complete sequence of a human genome

Sergey Nurk^{1†}, Sergey Koren^{1†}, Arang Rhie^{1†}, Mikko Rautiainen^{1†}, Andrey V. Bzikadze², Alla Mikheenko³, Mitchell R. Vollger⁴, Nicolas Altemose⁵, Lev Uralsky^{6,7}, Ariel Gershman⁸, Sergey Aganezov^{9†}, Savannah J. Hoyt¹⁰, Mark Diekhans¹¹, Glennis A. Logsdon⁴, Michael Alonge⁹, Stylianos E. Antonarakis¹², Matthew Borchers¹³, Gerard G. Bouffard¹⁴, Shelise Y. Brooks¹⁴, Gina V. Caldas¹⁵, Nae-Chyun Chen⁹, Haoyu Cheng^{16,17}, Chen-Shan Chin¹⁸, William Chow¹⁹, Leonardo G. de Lima¹³, Philip C. Dishuck⁴, Richard Durbin^{19,20}, Tatiana Dvorkina³, Ian T. Fiddes²¹, Giulio Formenti^{22,23}, Robert S. Fulton²⁴, Arkarachai Fungtammasan¹⁸, Erik Garrison^{11,25}, Patrick G. S. Grady¹⁰, Tina A. Graves-Lindsay²⁶, Ira M. Hall²⁷, Nancy F. Hansen²⁸, Gabrielle A. Hartley¹⁰, Marina Haukness¹¹, Kerstin Howe¹⁹, Michael W. Hunkapiller²⁹, Chirag Jain^{1,30}, Miten Jain¹¹, Erich D. Jarvis^{22,23}, Peter Kerpedjiev³¹, Melanie Kirsche⁹, Mikhail Kolmogorov³², Jonas Koriach²⁹, Milinn Kremitzki²⁶, Heng Li^{16,17}, Valerie V. Maduro³³, Tobias Marschall³⁴, Ann M. McCartney¹, Jennifer McDaniel³⁵, Danny E. Miller^{4,36}, James C. Mullikin^{14,28}, Eugene W. Myers³⁷, Nathan D. Olson³⁵, Benedict Paten¹¹, Paul Peluso²⁹, Pavel A. Pevzner³², David Porubsky⁴, Tamara Potapova¹³, Evgeny I. Rogaev^{6,7,38,39}, Jeffrey A. Rosenfeld⁴⁰, Steven L. Salzberg^{9,41}, Valerie A. Schneider⁴², Fritz J. Sedlazeck⁴³, Kishwar Shafin¹¹, Colin J. Shew⁴⁴, Alaina Shumate⁴¹, Ying Sims¹⁹, Arian F. A. Smit⁴⁵, Daniela C. Soto⁴⁴, Ivan Sovic^{29,46}, Jessica M. Storer⁴⁵, Aaron Streets^{5,47}, Beth A. Sullivan⁴⁸, Françoise Thibaud-Nissen⁴², James Torrance¹⁹, Justin Wagner³⁵, Brian P. Walenz¹, Aaron Wenger²⁹, Jonathan M. D. Wood¹⁹, Chunlin Xiao⁴², Stephanie M. Yan⁴⁹, Alice C. Young¹⁴, Samantha Zarate⁹, Urvashi Surti⁵⁰, Rajiv C. McCoy⁴⁹, Megan Y. Dennis⁴⁴, Ivan A. Alexandrov^{3,7,51}, Jennifer L. Gerton^{13,52}, Rachel J. O'Neill¹⁰, Winston Timp^{8,41}, Justin M. Zook³⁵, Michael C. Schatz^{9,49}, Evan E. Eichler^{4,53*}, Karen H. Miga^{11,54*}, Adam M. Phillippy^{1*}

Since its initial release in 2000, the human reference genome has covered only the euchromatic fraction of the genome, leaving important heterochromatic regions unfinished. Addressing the remaining 8% of the genome, the Telomere-to-Telomere (T2T) Consortium presents a complete 3.055 billion–base pair sequence of a human genome, T2T-CHM13, that includes gapless assemblies for all chromosomes except Y, corrects errors in the prior references, and introduces nearly 200 million base pairs of sequence containing 1956 gene predictions, 99 of which are predicted to be protein coding. The completed regions include all centromeric satellite arrays, recent segmental duplications, and the short arms of all five acrocentric chromosomes, unlocking these complex regions of the genome to variational and functional studies.

Lander et al. (2001) *Nature*
 Venter et al. (2001) *Science*
 Nurk et al. (2022) *Science*

“Landmark human genome papers”

<https://doe-humangenomeproject.ornl.gov/landmark-hgp-papers/>

The Critical Question

- How can we exploit the high expected degree of similarity to accelerate the search?
- Key considerations:
 - DNA is a four-letter alphabet, there will be lots of **redundant sequences** in the genome
 - We can use approaches that treat mismatches and gaps less efficiently than dynamic programming

Preliminaries: strings

Offsets

S ← AGGCCTA

0123456

S[3] = C

S[5:] = TA

Substring

AGCGCTA

AGCGCTA

GCGC

CTA

A

GC

GC

Terminator

AGCGCTA\$

\$ < A < C < G < T

AA\$ < AAA\$

BEE < BEER

Preliminaries: strings

Prefixes

AGCGCTA\$

AGCGCTA

AGCGCT

AGCGC

AGCG

AGC

AG

A

Suffixes

AGCGCTA\$

AGCGCTA\$

GCGCTA\$

CGCTA\$

GCTA\$

CTA\$

TA\$

A\$

\$

Naive Searching

Find places where *pattern* P occurs as a substring of *text* T .
Each such place is an *occurrence* or *match*.

Let $n = |P|$, and let $m = |T|$ Assume $n \leq m$

Alignment: a way of putting P 's characters opposite T 's.
May or may not correspond to an match.

P : word

T : There would have been a time for such a word

Alignment 1: word

Alignment 2: word

Naive Searching

P: word

T: There would have been a time for such a word

word word word word word word word word word word word

word word word word word word word word word word

word word word word word word word word word word

word word word word word word word word word word

word word word word word word word word word word

One
occurrence



Try all possible alignments. For each, check if it matches.

This is the *naïve algorithm*.

Not good: back to $m - n + 1$ possible matches or $\sim 4^n$ with gaps!

Skip fruitless alignments: Boyer-Moore

P: word

T: There would have been a time for such a word

.....word----->
 ->

u doesn't occur in *P*, so skip next two alignments

P: word

T: There would have been a time for such a word

.....word----->
 word skip!
 word skip!
 word

extending to
approximate
matching is
awkward.

Scaling in genome size will not work!

P : word

T : There would have been a time for such a word

-----word----->

As m & n grow, # characters comparisons grows with...

$|P| = n$ $|T| = m$

	Naïve matching	Boyer-Moore
Worst case	$m \cdot n$	m
Best case	m	m / n

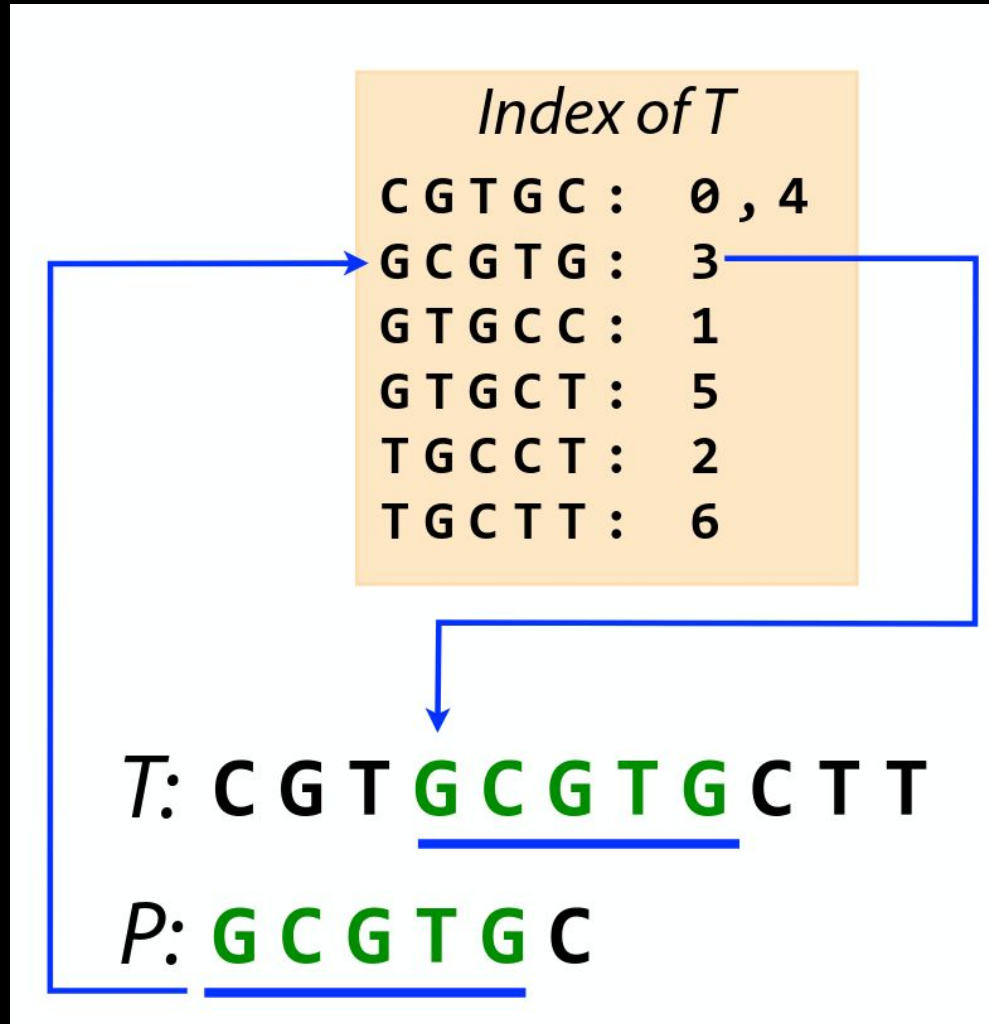
Read mapping:

- Many short reads (small n) vs.
- Large genome (big m).
- Multiple matches possible

BLAST has same issue

Can we use some indexing tricks to scale in n rather than m ?

K-mer Indexing



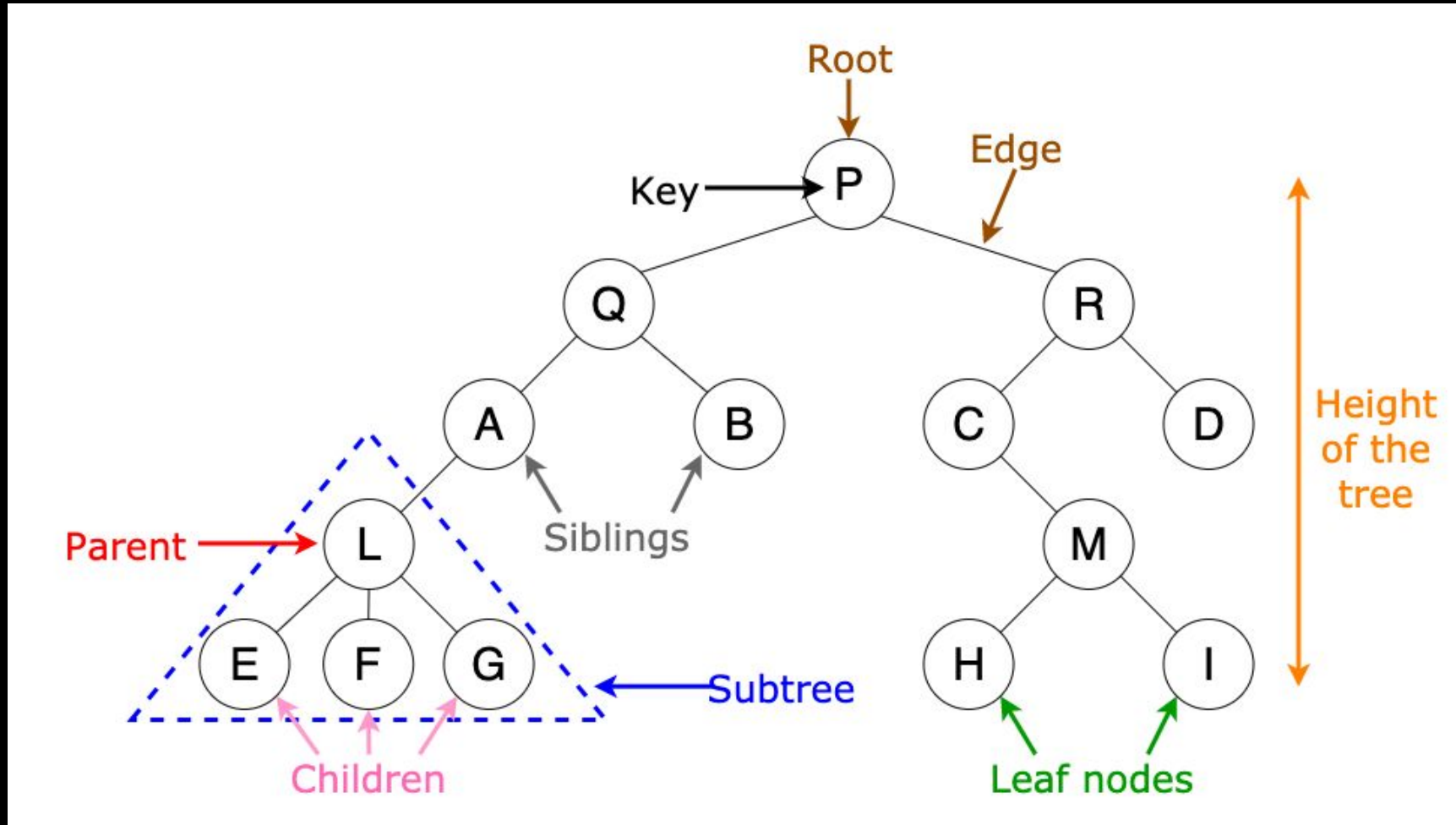
Reference indexed once and $O(1)$ lookup

BUT:

- K-mers shorter than reads
- Extensions requires many lookups (neighbourhood enumeration) or expensive DP
- Repetitive K-mers mean many candidates
- Index doesn't easily capture context (adjacent K-mers)

Let's try a different way
of indexing a genome

Trees in Computer Science



A trie is a special type of tree

Keys: instant, internal, internet

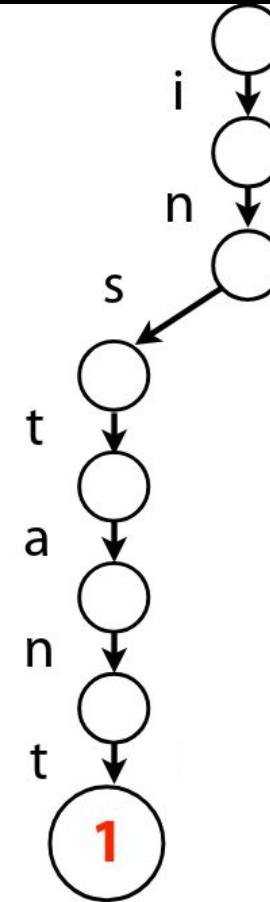
Key	Value
instant	1
internal	2
internet	3

Smallest tree such that:

Each edge is labeled with a character $c \in \Sigma$

For given node, at most one child edge has label c , for any $c \in \Sigma$

Each key is “spelled out” along some path starting at root



A trie is a special type of tree

Keys: instant, internal, internet

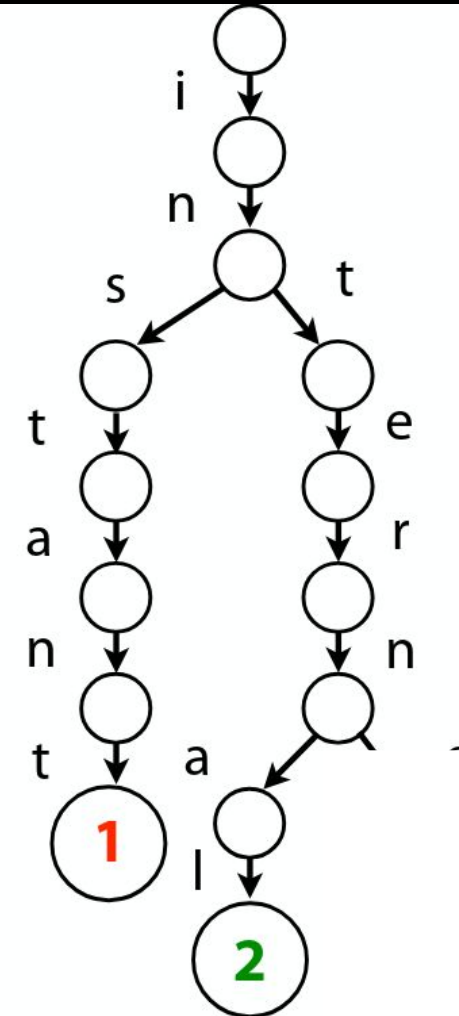
Key	Value
instant	1
internal	2
internet	3

Smallest tree such that:

Each edge is labeled with a character $c \in \Sigma$

For given node, at most one child edge has label c , for any $c \in \Sigma$

Each key is “spelled out” along some path starting at root



A trie is a special type of tree

Keys: instant, internal, internet

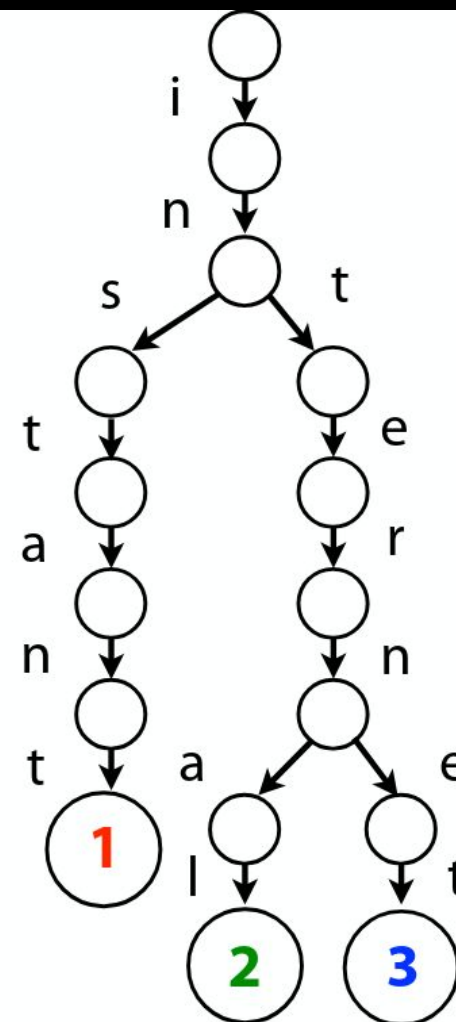
Key	Value
instant	1
internal	2
internet	3

Smallest tree such that:

Each edge is labeled with a character $c \in \Sigma$

For given node, at most one child edge has label c , for any $c \in \Sigma$

Each key is “spelled out” along some path starting at root



Suffix trie: trie containing all suffixes

T = aba\$

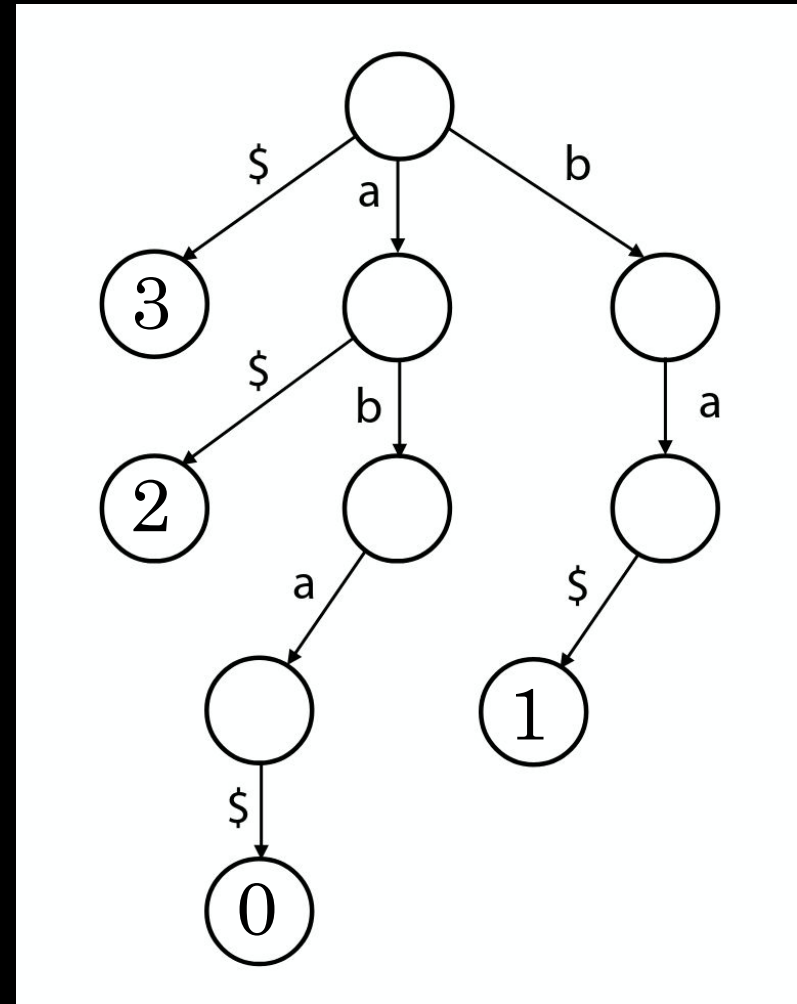
aba\$

ba\$

a\$

\$

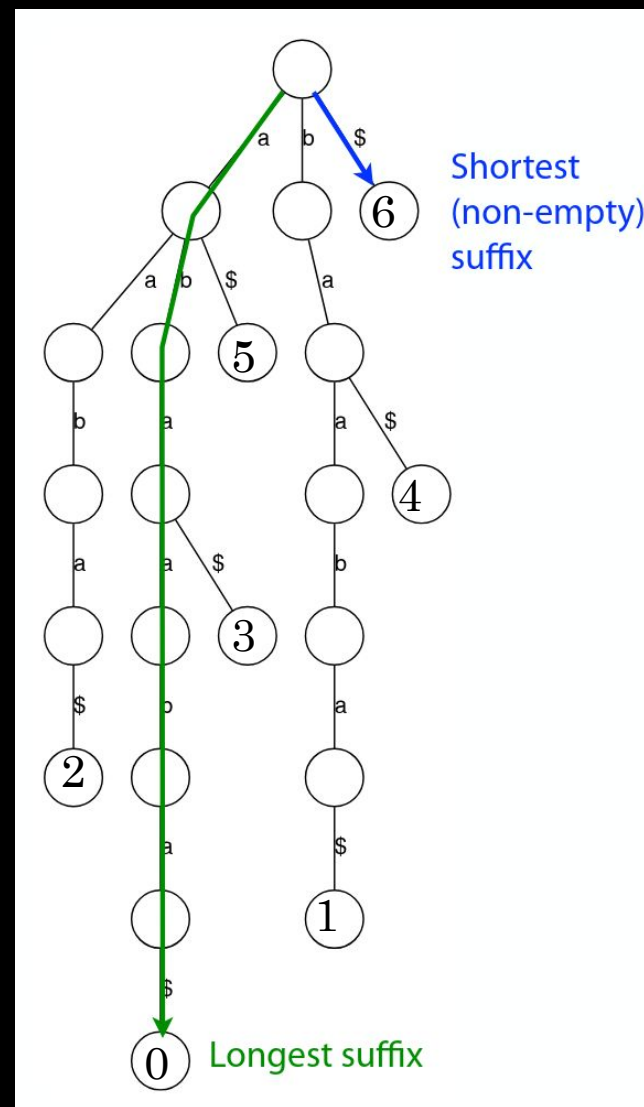
Each path from root to leaf represents a suffix; each suffix is represented by some path from root to leaf



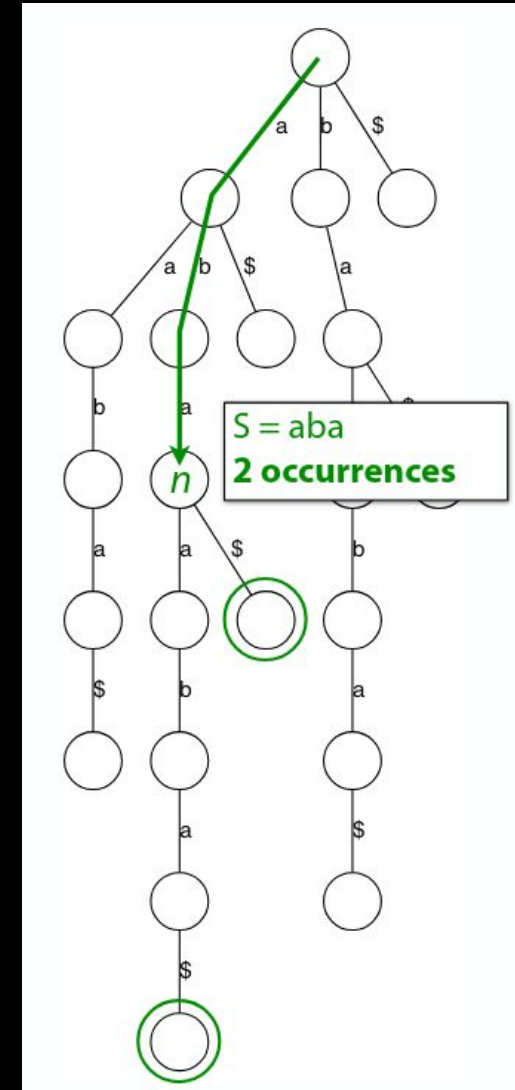
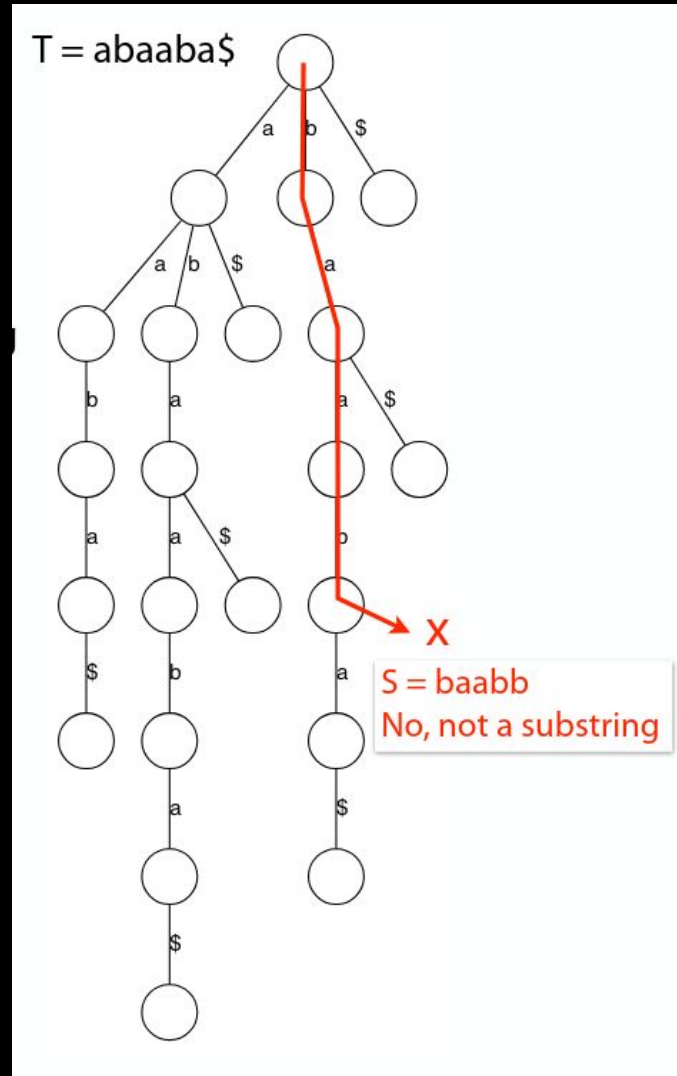
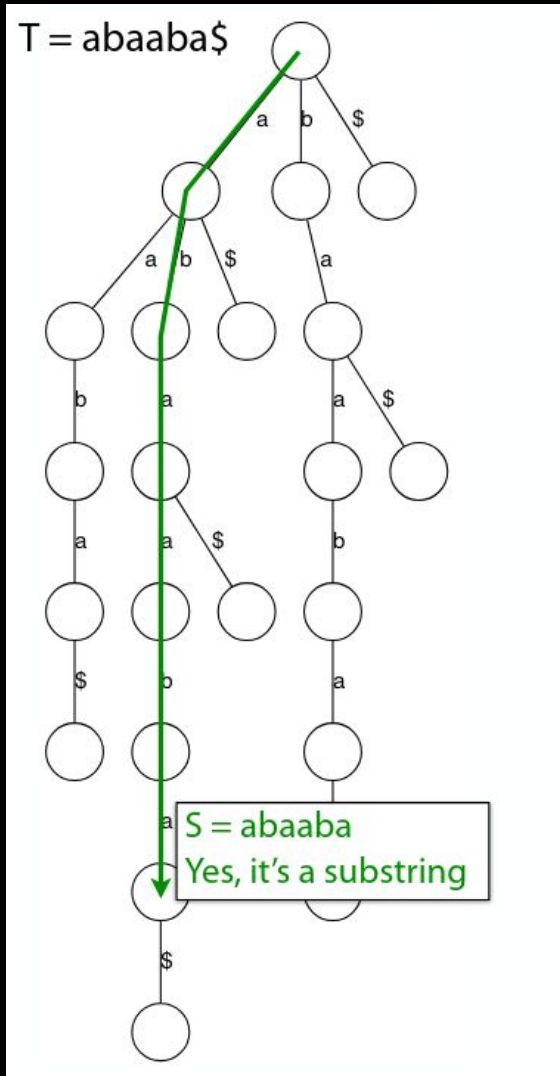
Suffix trie: trie containing all suffixes

T = abaaba\$
abaaba\$
baaba\$
aaba\$
aba\$
ba\$
a\$
\$

Each path from root to leaf represents a suffix; each suffix is represented by some path from root to leaf

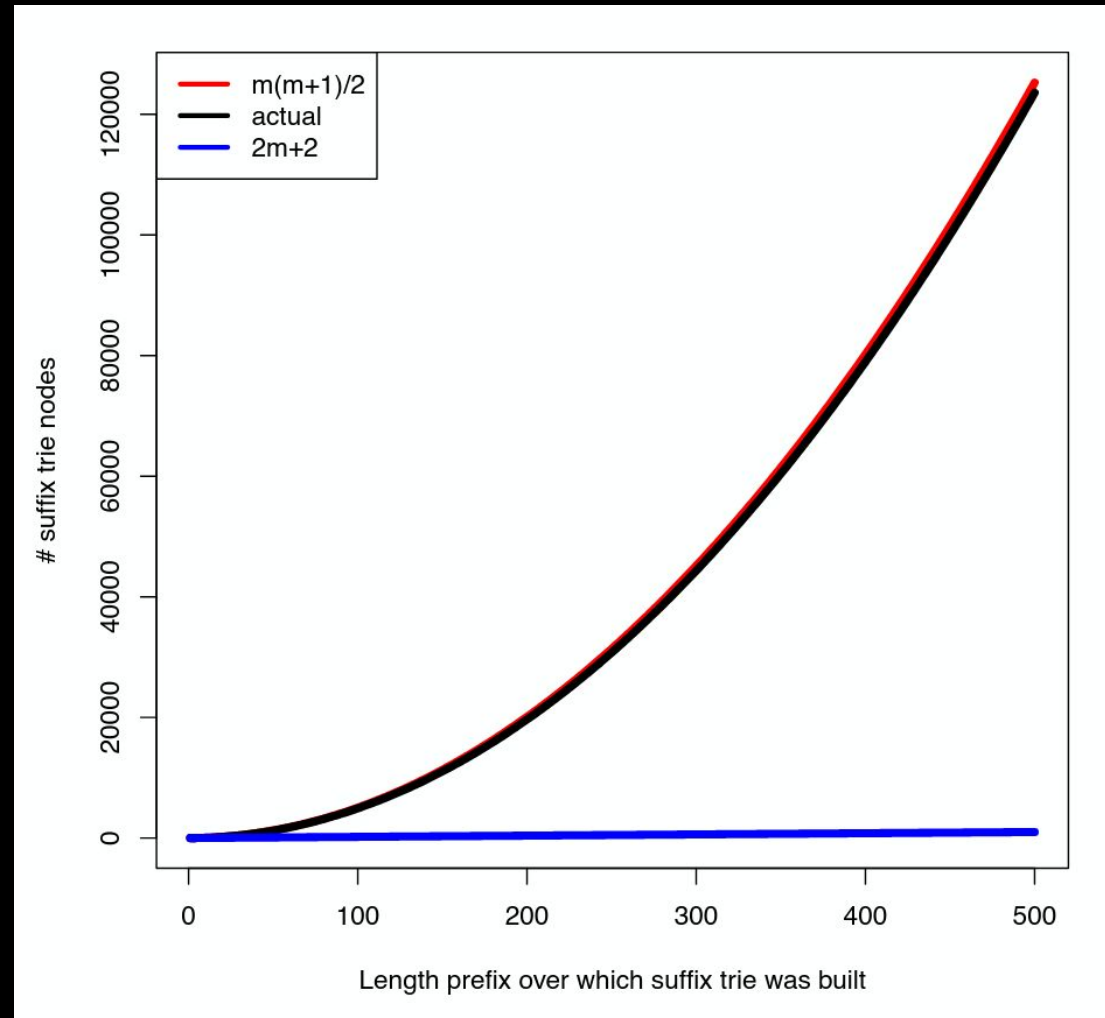


Why? Search and count in $O(|S|)$

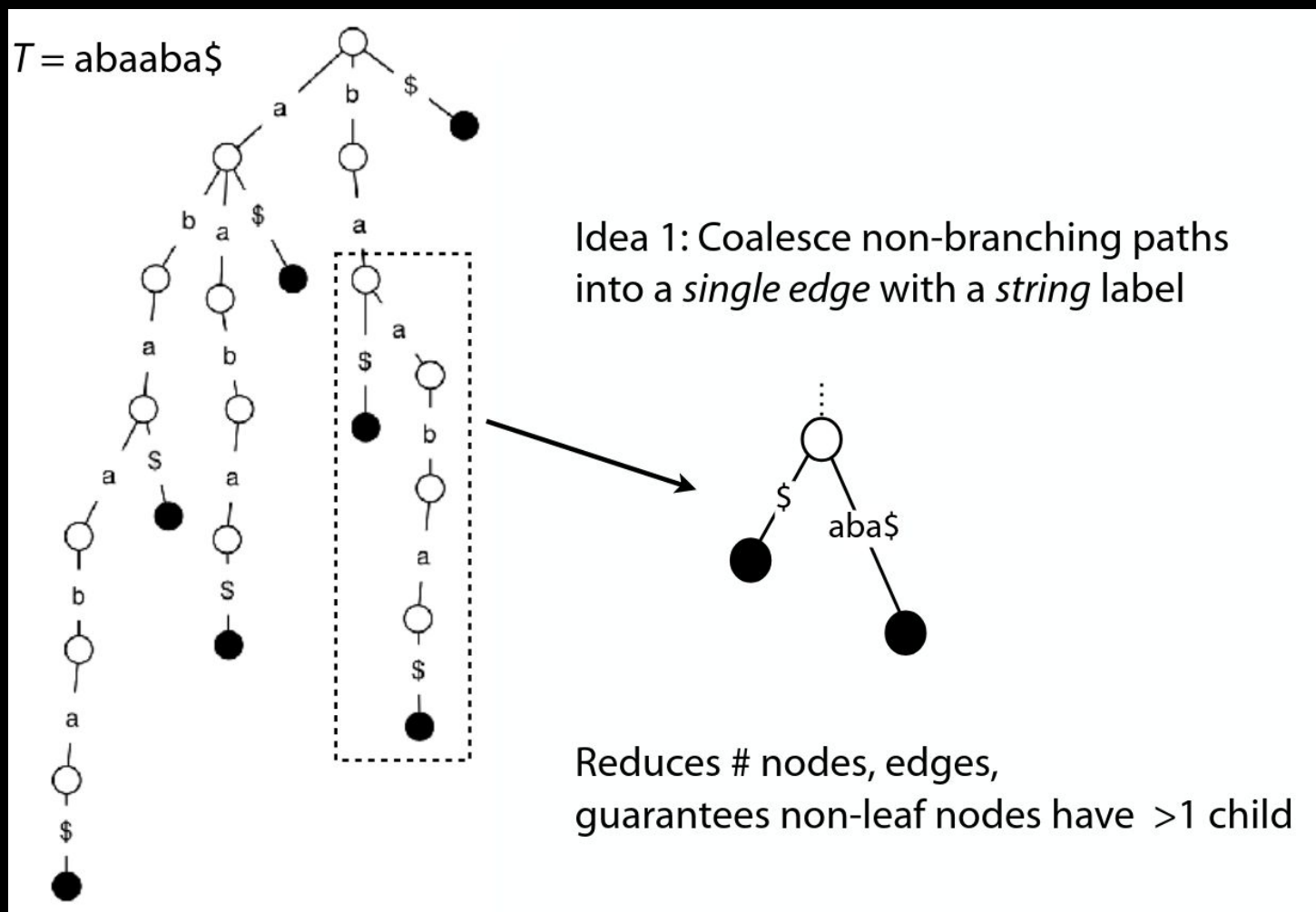


But Suffix Trie $\sim O(m^2 / 2)$ space

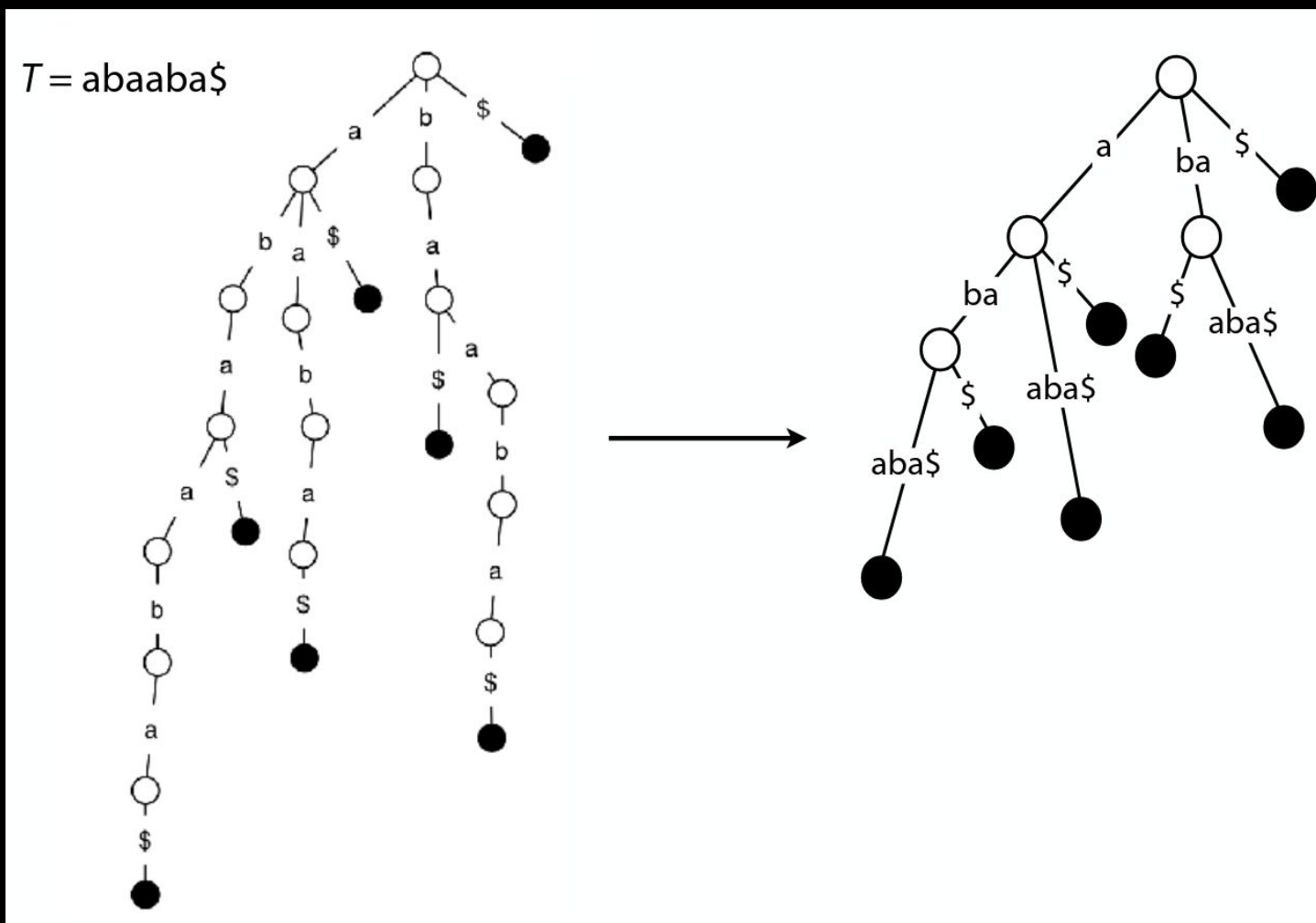
T = abaaba\$
abaaba\$
baaba\$
aaba\$
aba\$
ba\$
a\$
\$



Compressing a Suffix Trie into a Tree



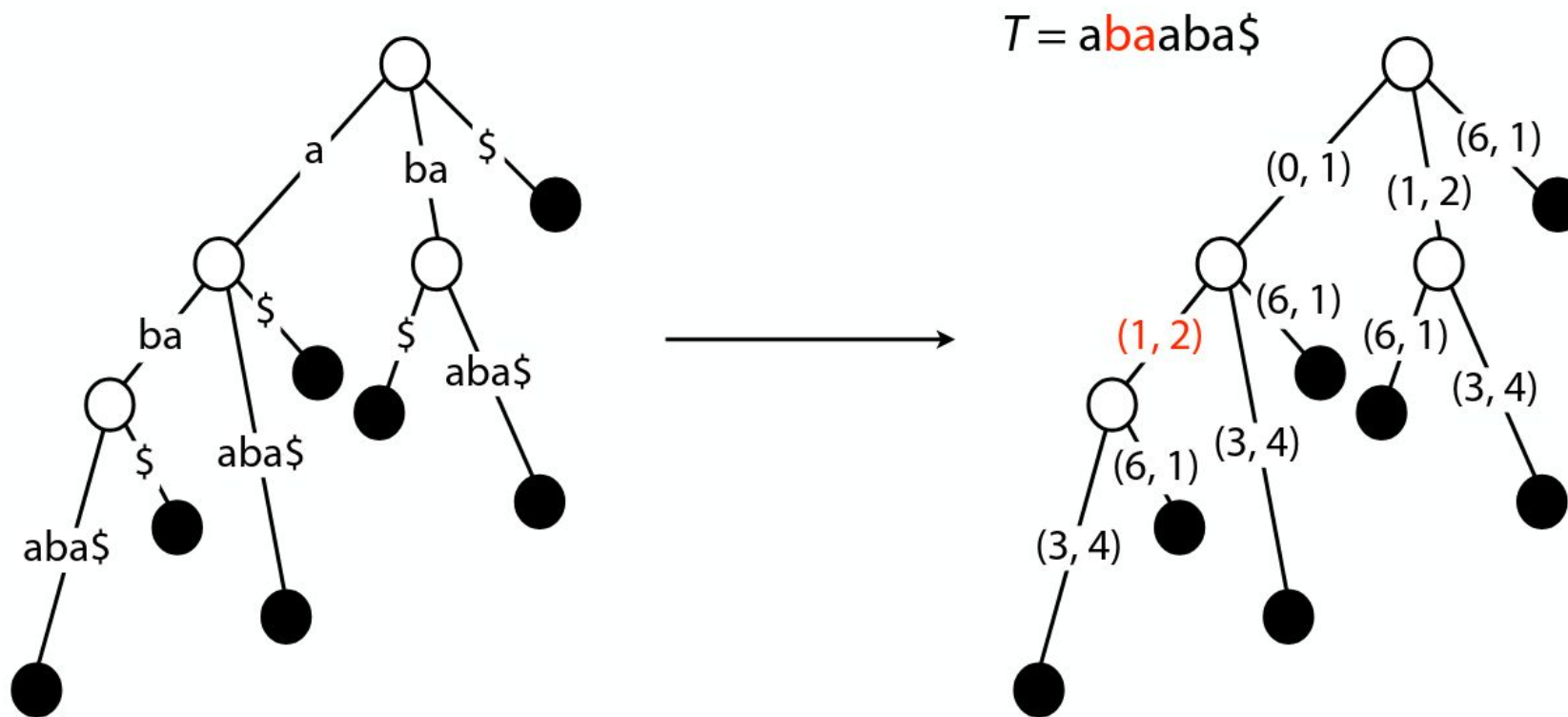
Compressing a Suffix Trie into a Tree



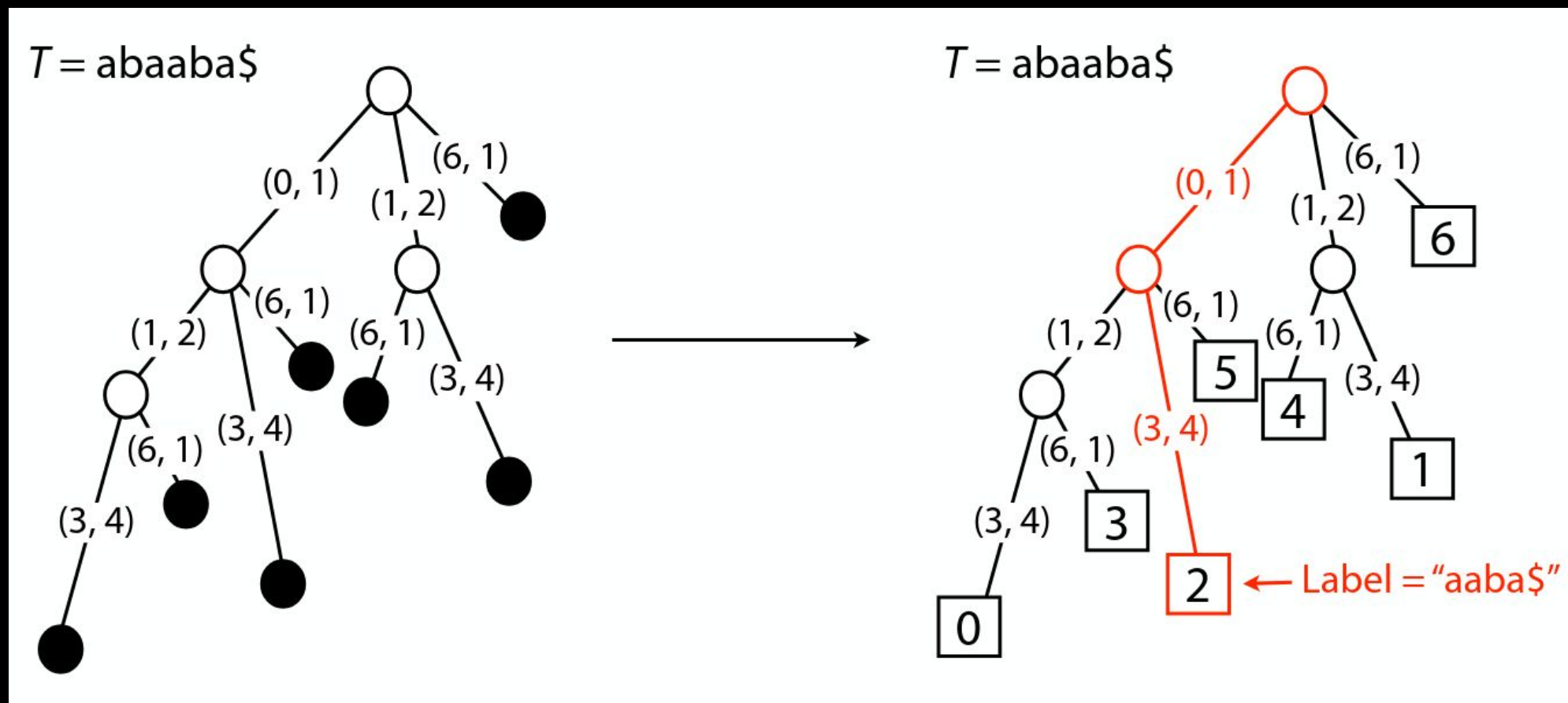
Storing suffix strings still memory intensive

Storing locations instead of strings

Idea 2: Store T itself in addition to the tree. Convert tree's edge labels to (offset, length) pairs with respect to T .



Label leaves with offsets



Retain the same operations

Suffix Trie $\sim O(m^2/2)$ space

Suffix Tree $\sim O(m)$ space!

BUT... don't forget constant factors!

MUMmer ≈ 15.76 bytes per nt

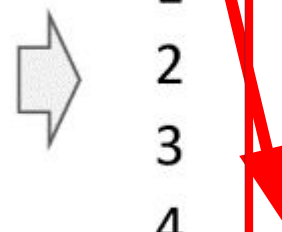
Suffix tree of human genome will still be $>45\text{GB}$ (or larger depending on exact data structures underlying suffix tree nodes/edges)

	Suffix tree
Time: Does P occur?	$O(n)$
Time: Count k occurrences of P	$O(n + k)$
Time: Report k locations of P	$O(n + k)$
Space	$O(m)$

$m = |T|$, $n = |P|$, $k = \#$ occurrences of P in T

From suffix trees to suffix arrays

Idx	Suffixes	SA-Idx	Idx	Sorted Suffix
0	BANANA\$	0	6	\$
1	ANANA\$	1	5	A\$
2	NANA\$	2	3	ANA\$
3	ANA\$	3	1	ANANA\$
4	NA\$	4	0	BANANA\$
5	A\$	5	4	NA\$
6	\$	6	2	NANA\$



Every suffix, sorted lexicographically
(\$ is smallest)

Index value remains the same!

What is the point of *this*?

Idx	Suffixes	SA-Idx	Idx	Sorted Suffix
0	BANANA\$	0	6	\$
1	ANANA\$	1	5	A\$
2	NANA\$	2	3	ANA\$
3	ANA\$	3	1	ANANA\$
4	NA\$	4	0	BANANA\$
5	A\$	5	4	NA\$
6	\$	6	2	NANA\$

⇒

↓

Suffix Array [6, 5, 3, 1, 0, 4, 2]

We now have a compressed representation where lexicographically similar strings **are adjacent to one another**

All suffixes that start with “A” are at positions 1 to 3 (because the first position is numbered 0) in the suffix array

So if we’re searching for “A” we only need to look at this **consecutive block of rows!**

Still $O(m)$ space BUT much reduced constant factor $\sim 4 \text{ bytes/base} * 3 \text{bn bp} \approx 12 \text{ GB}$.
Suffix tree is $>45 \text{ GB}$.

Does my sequence match?

Search “ANA” against the suffix array

Idx	Suffixes	SA-Idx	Idx	Sorted Suffix
0	BANANA\$	0	6	\$
1	ANANA\$	1	5	A\$
2	NANA\$	2	3	ANA\$
3	ANA\$	3	1	ANANA\$
4	NA\$	4	0	BANANA\$
5	A\$	5	4	NA\$
6	\$	6	2	NANA\$

Suffix Array [6, 5, 3, 1, 0, 4, 2]

Binary search to find first “ANA”

Progress to last “ANA”

There are two matches of “ANA” in “BANANA”

They start at positions 3 and 1

Naive: $O(n \log m)$, Skipping: $O(n + \log m)$ - smaller but slower than suffix array!

BUT... still a large index
and inexact matching (not
covered) is a bit painful!

Want something smaller than suffix array but with query
time of suffix tree (and can handle inexact matches)

Burrows-Wheeler Transform

All Rotations

B A N A N A \$
A N A N A \$ B
N A N A \$ B A
A N A \$ B A N
N A \$ B A N A
A \$ B A N A N
\$ B A N A N A

Sort Rotations

\$ B A N A N A
A \$ B A N A N
A N A \$ B A N
A N A N A \$ B
B A N A N A \$
N A \$ B A N A
N A N A \$ B A

$\$ < a < b < n$

Take last column

\$ B A N A N A
A \$ B A N A N
A N A \$ B A N
A N A N A \$ B
B A N A N A \$
N A \$ B A N A
N A N A \$ B A

$\text{BWT}(\text{BANANA}\$) = \text{ANNB}\$ \text{AA}$

Why Do Such a Ridiculous Thing?

F		L
\$	BANANA	A
A	\$BANAN	
A	NA\$BAN	
A	NANA\$B	
B	BANANA	\$
N	A\$BANA	
N	ANA\$BA	

If we're doing string searching, we need to explore different rows of the matrix

The end of one row should point to the row that starts with the **same exact character** in the string. This is LF mapping

Why Do Such a Ridiculous Thing?

F	L
\$BANANA	A
A\$BANAN	
ANA\$BAN	
ANANA\$B	
BANANA\$	
NA\$BANA	
NANA\$BA	

The first “A” in L is also the first “A” in F

Why Do Such a Ridiculous Thing?

F		L
\$BANANA	A	
A\$BANAN		N
ANA\$BAN		
ANANA\$B		
BANANA\$		
N	A\$BANA	
NANA\$BA		

The first “N” in L is also the first “N” in F

LF-Mapping allows reversal of BWT

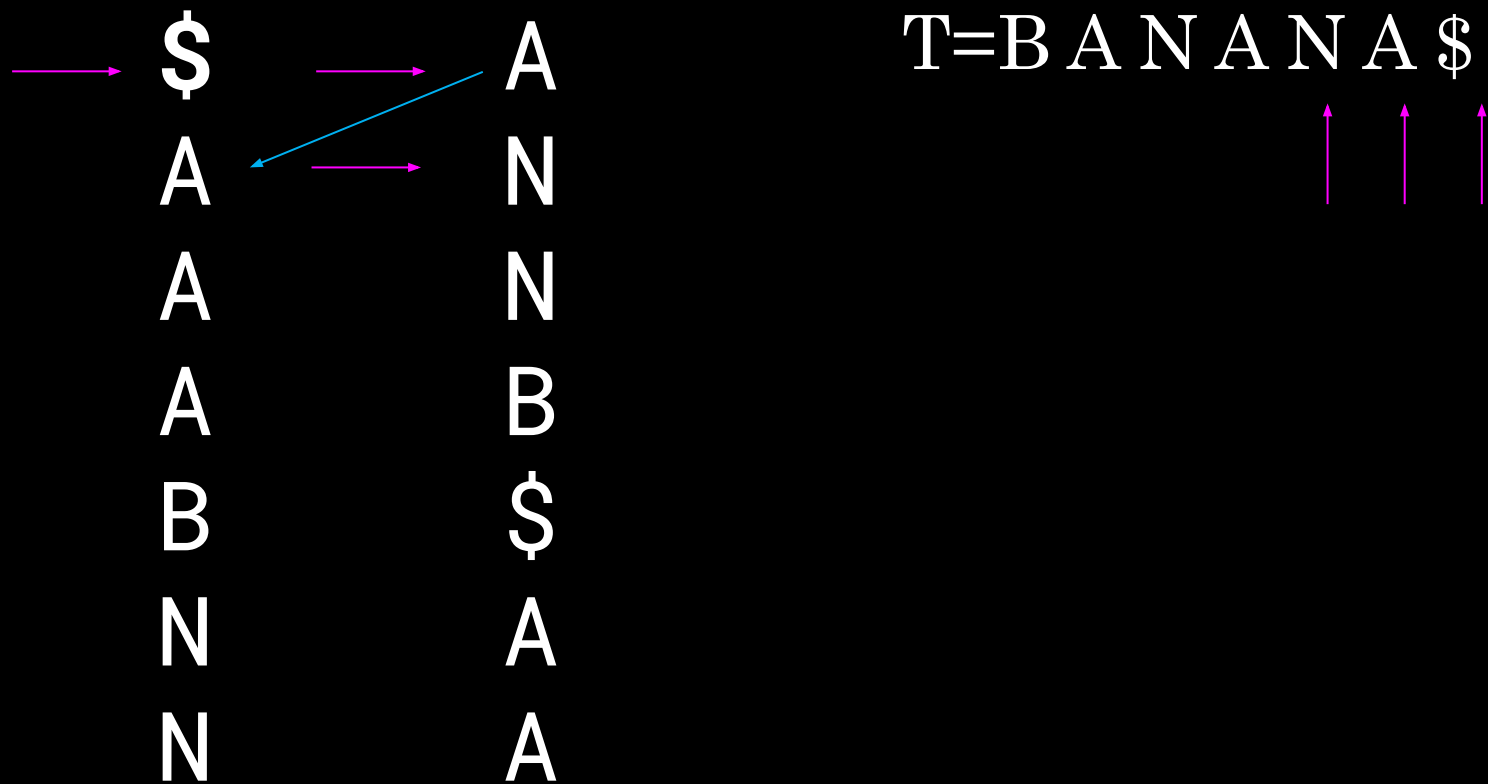
\$BANANA
A\$BANAN
ANA\$BAN
ANANA\$B
BANANA\$
NA\$BANA
NANA\$BA

LF-Mapping allows reversal of BWT

→	\$	→	A		T=B A N A N A \$
	A		N		↑ ↑
	A		N		
	A		B		
	B		\$		
	N		A		
	N		A		

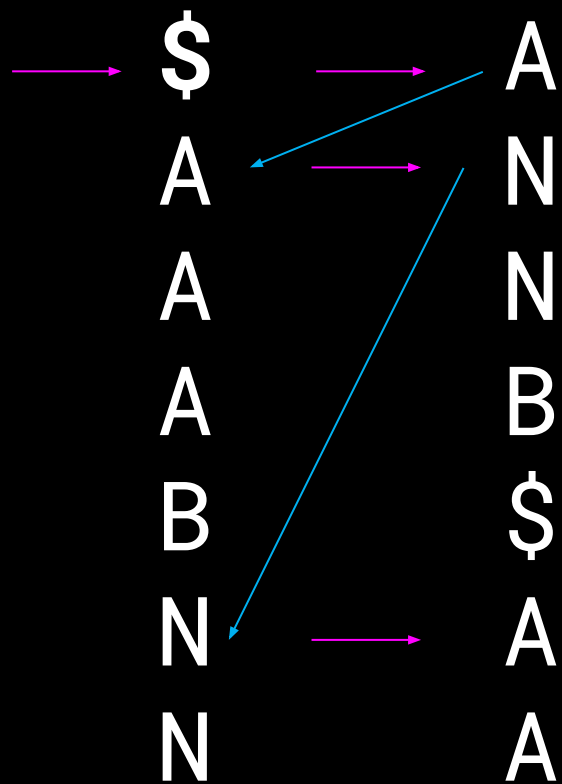
BWT(T)=ANNB\$AA

LF-Mapping allows reversal of BWT

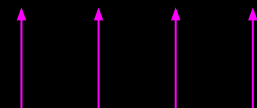


BWT(T) = ANNB\$AA

LF-Mapping allows reversal of BWT

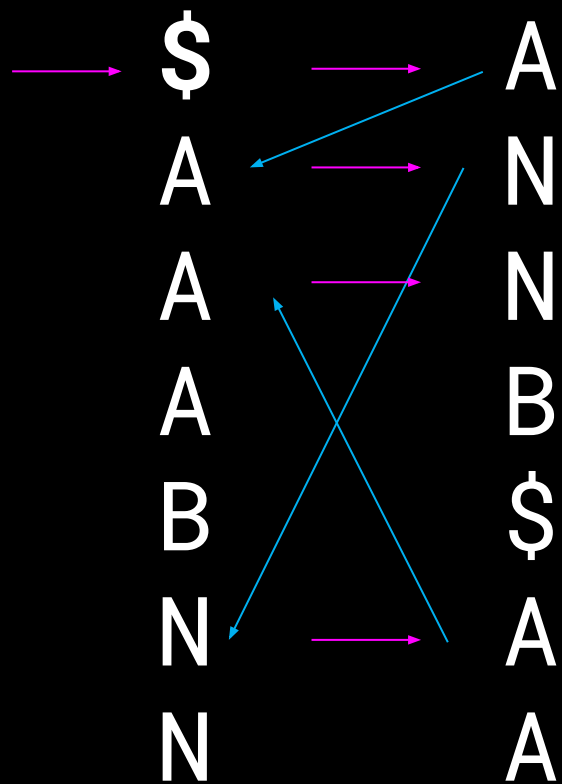


$T = \text{BANANA\$}$

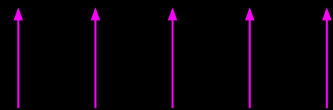


$\text{BWT}(T) = \text{ANNB\$AA}$

LF-Mapping allows reversal of BWT

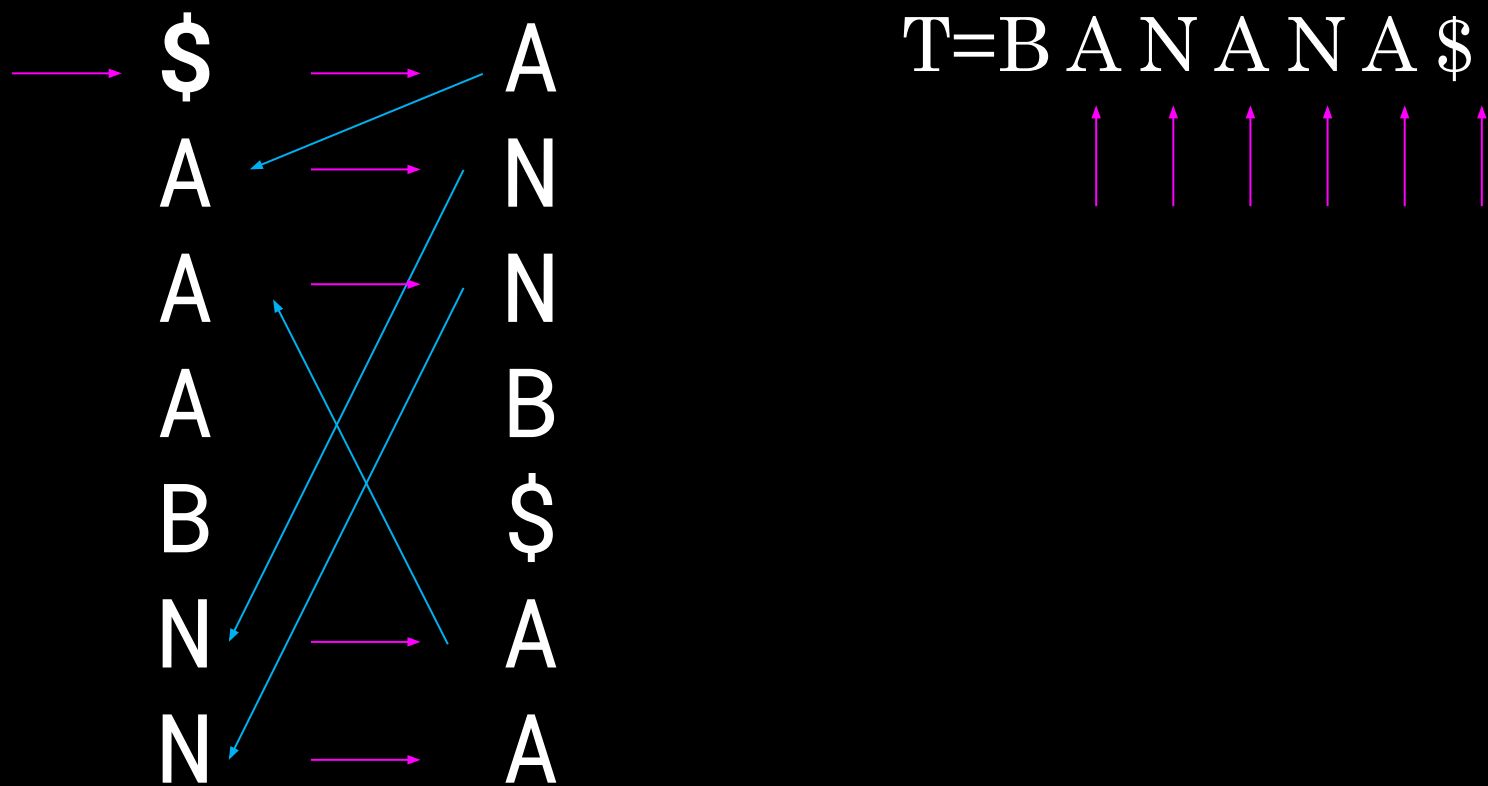


T=BANANA\$



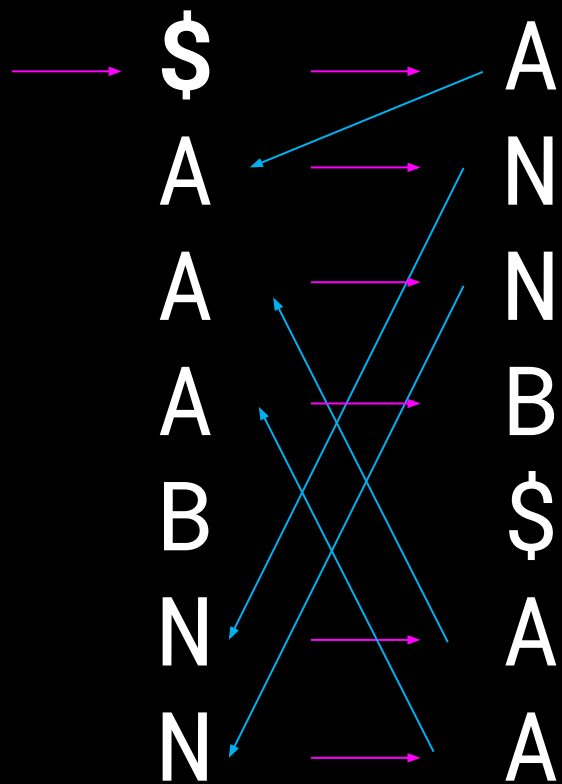
BWT(T)=ANNB\$AA

LF-Mapping allows reversal of BWT



BWT(T)=ANNB\$AA

LF-Mapping allows reversal of BWT



T=B A N A N A \$

↑ ↑ ↑ ↑ ↑ ↑ ↑

BWT(T)=ANNB\$AA

Searching BWM for a string

Look for range of rows of BWM(T) with P as prefix

Start with shortest suffix, then match successively longer suffixes

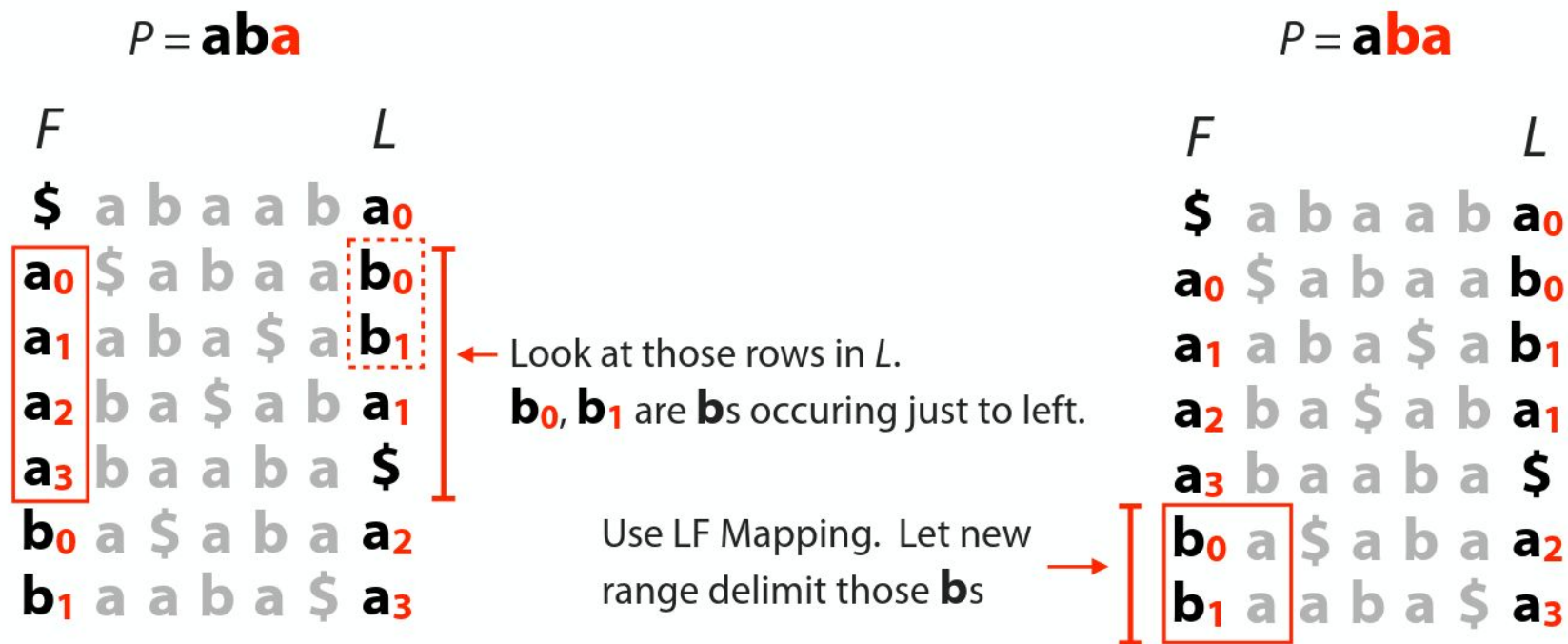
$P = \mathbf{ab}\mathbf{a}$

Easy to find all the
rows beginning with $\mathbf{a}$

F						L
\$	a	b	a	a	b	$\mathbf{a_0}$
$\mathbf{a_0}$	\$	a	b	a	a	$\mathbf{b_0}$
$\mathbf{a_1}$	a	b	a	\$	a	$\mathbf{b_1}$
$\mathbf{a_2}$	b	a	\$	a	b	$\mathbf{a_1}$
$\mathbf{a_3}$	b	a	a	b	a	\$
$\mathbf{b_0}$	a	\$	a	b	a	$\mathbf{a_2}$
$\mathbf{b_1}$	a	a	b	a	\$	$\mathbf{a_3}$

Searching BWM for a string

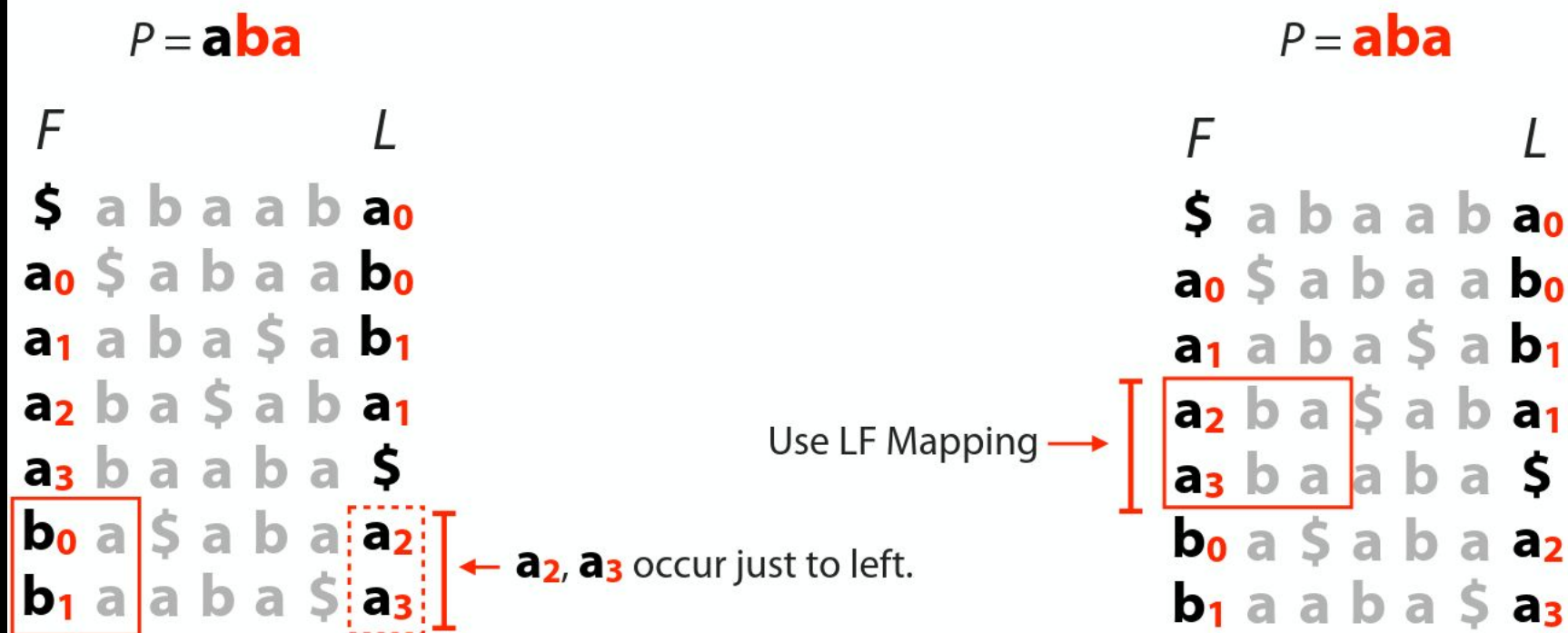
We have rows beginning with **a**, now we want rows beginning with **ba**



Now we have the rows with prefix **ba**

Searching BWM for a string

We have rows beginning with **ba**, now we seek rows beginning with **aba**



Now we have the rows with prefix **aba**

Searching BWM for a string

$P = \mathbf{aba}$

Got the same range, $[3, 5)$, we would have got from suffix array

	F		L
	\$	a b a a b	a₀
	a₀	\$ a b a a	b₀
	a₁	a b a \$ a	b₁
[3, 5)	a₂	b a \$ a b	a₁
	a₃	b a a b a	\$
	b₀	a \$ a b a	a₂
	b₁	a a b a \$	a₃

Where are these?

6	\$
5	a \$
2	a a b a \$
3	a b a \$
0	a b a a b a \$
4	b a \$
1	b a a b a \$

[3, 5)

Unlike suffix array, we don't immediately know *where* the matches are in T...

Searching BWM for a string

When P does not occur in T , we eventually fail to find next character in L :

$P = \mathbf{bba}$

F						L	
\$	a	b	a	a	b	a_0	
a_0	\$	a	b	a	a	b_0	
a_1	a	b	a	\$	a	b_1	
a_2	b	a	\$	a	b	a_1	
a_3	b	a	a	b	a	\$	
Rows with ba prefix	b_0	a	\$	a	b	a	a_2
	b_1	a	a	b	a	\$	a_3

 No **bs**!

BWA:

The Burrows-Wheeler Aligner

Idx	Sorted Suffix		
6	\$	\$BANAN	A
5	A\$	A\$BANA	N
3	ANA\$	ANA\$BA	N
1	ANANA\$	ANANA\$	B
0	BANANA\$	BANANA	\$
4	NA\$	NA\$BAN	A
2	NANA\$	NANA\$B	A

Rapid string search using the **sorted index** (equivalent to F) and the **BWT string** (equal to L)

BWA: The Burrows-Wheeler Aligner

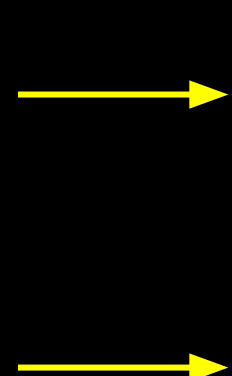
Idx	Sorted Suffix
6	\$
5	A\$
3	ANA\$
1	ANANA\$
0	BANANA\$
4	NA\$
2	NANA\$

\$BANAN	A
A\$BANA	N
ANA\$BA	N
ANANA\$	B
BANANA	\$
NA\$BAN	A
NANA\$B	A

Key parts
of an
FM-Index

Rapid string search using the **sorted index**
(equivalent to F) and the **BWT string** (equal to L)
And nothing else

Why this is awesome: Sequence reads are effectively searched against different parts of the reference genome at the same time



0	6	\$googo l
1	3	gol\$go o
2	0	googol \$
3	5	l\$goog o
4	2	ogol\$g o
5	4	ol\$goo g
6	1	oogol\$ g

Using the L-F mapping allows us to **remove redundancy** relative to the suffix array

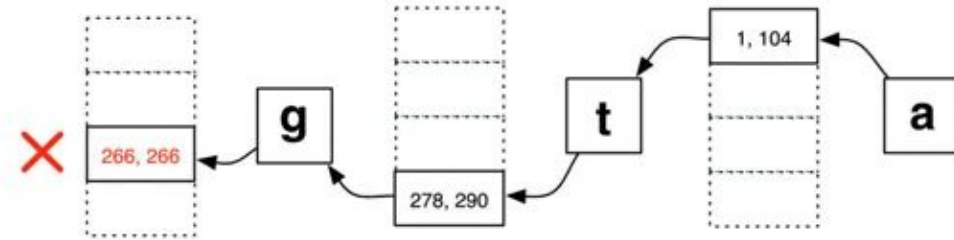
- **Why this is slightly less awesome:** Preprocessing requires many GB of memory
- What about mismatches?

Backtracking

Search string: GGTA

“Genome” contains no
GGTA, but contains GTTG

Exact



BWA refinements

Allowing mismatches: maximum deviation from the search string

- Store searches in a heap to prioritize the lowest mismatches in the search so far

Custom penalties for mismatches, insertion and deletions

Double indexing: BWTs from both ends meet in the middle (avoids massive amounts of futile backtracing)

Memory refinements: store only parts of the BWT and O matrix, calculate the rest on the fly

Suffix tree vs Suffix array vs FM-index

	Suffix tree	Suffix array	FM Index
Time: Does P occur?	$O(n)$	$O(n \log m)$	$O(n)$
Time: Count k occurrences of P	$O(n + k)$	$O(n \log m)$	$O(n)$
Time: Report k locations of P	$O(n + k)$	$O(n \log m + k)$	$O(n + k)$
Space	$O(m)$	$O(m)$	$O(m)$
Needs T ?	yes	yes	no
Bytes per input character	>15	~4	~0.5

$$m = |T|, n = |P|, k = \# \text{ occurrences of } P \text{ in } T$$

Program	Single-end			Paired-end		
	Time (s)	Conf (%)	Err (%)	Time (s)	Conf (%)	Err (%)
bowtie-125	1966	88.0	0.07	1701	91.0	0.37
BWA-125	3021	93.0	0.05	3059	97.6	0.04
MAQ-125	17506	92.7	0.08	19388	96.3	0.02
SOAP2-125	555	91.5	0.17	1187	90.8	0.14

One million pairs of 32, 70 and 125 bp reads, respectively, were simulated from the human genome with 0.09% SNP mutation rate, 0.01% indel mutation rate and 2% uniform sequencing base error rate. The insert size of 32 bp reads is drawn from a normal distribution $N(170,25)$, and of 70 and 125 bp reads from $N(500,50)$. CPU time in seconds on a single core of a 2.5 GHz Xeon E5420 processor (Time), percent confidently mapped reads (Conf) and percent erroneous alignments out of confident mappings (Err) are shown in the table.

SOAP2: VERY approximately 1000x faster than BLAST

Table 2. Evaluation on real data

Program	Time (h)	Conf (%)	Paired (%)
Bowtie	5.2	84.4	96.3
BWA	4.0	88.9	98.8
MAQ	94.9	86.1	98.7
SOAP2	3.4	88.3	97.5

The 12.2 million read pairs were mapped to the human genome. CPU time in hours on a single core of a 2.5 GHz Xeon E5420 processor (Time), percent confidently mapped reads (Conf) and percent confident mappings with the mates mapped in the correct orientation and within 300 bp (Paired), are shown in the table.

The

